

student

January 15, 2026

1 Prédiction de la Performance Étudiante : Architecture Big Data avec Spark et GCP

Ce devoir a pour objectif de concevoir un pipeline complet de Data Engineering et de Machine Learning capable de prendre en charge d'énorme volume de données et de les traiter. En utilisant Apache Spark sur Google Cloud Platform, nous analysons les facteurs influençant la réussite scolaire pour prédire un index de performance.

Points clés du projet :

Ingestion : Lecture distribuée depuis Google Cloud Storage.

Qualité des données : Analyse statistique et détection d'outliers avec Spark SQL.

Machine Learning : Implémentation d'un modèle Random Forest via un Pipeline MLlib.

Architecture Cloud : Export des résultats vers BigQuery pour une visualisation décisionnelle.

```
[224]: from pyspark.sql import SparkSession
       from pyspark.sql.functions import col
```

```
spark = SparkSession.builder \
    .appName("Student_Performance_Final_Project") \
    .getOrCreate()
```

```
[225]: # Chargement
data_path = "gs://student-performance-exam/StudentPerformance.csv"
df_raw = spark.read.csv(data_path, header=True, inferSchema=True)
```

```
[226]: # On remplace les espaces par des underscores dès le début
df = df_raw.select([col(c).alias(c.replace(' ', '_')) for c in df_raw.columns])

df.printSchema()
```

```
root
|-- Hours_Studied: integer (nullable = true)
|-- Previous_Scores: integer (nullable = true)
|-- Extracurricular_Activities: string (nullable = true)
```

```

|-- Sleep_Hours: integer (nullable = true)
|-- Sample_Question_Papers_Practiced: integer (nullable = true)
|-- Performance_Index: double (nullable = true)

```

```
[227]: df.show(5)
```

```

+-----+-----+-----+-----+-----+
|Hours_Studied|Previous_Scores|Extracurricular_Activities|Sleep_Hours|Sample_Que
stion_Papers_Practiced|Performance_Index|
+-----+-----+-----+-----+-----+
|          7|          99|          Yes|          9|
1|          91.0|
|          4|          82|          No|          4|
2|          65.0|
|          8|          51|          Yes|          7|
2|          45.0|
|          5|          52|          Yes|          5|
2|          36.0|
|          7|          75|          No|          8|
5|          66.0|
+-----+-----+-----+-----+-----+
only showing top 5 rows

```

2 EDA

```
[228]: import pyspark.sql.functions

# On parcourt chaque colonne et on compte les valeurs nulles
df.select([sum(when(col(c).isNull(), 1).otherwise(0)).alias(c) for c in df.
    ↪columns]).show()

# on vérifie aussi si des chaînes de caractères sont vides
print("chaînes vides :")
df.filter(col("Extracurricular_Activities") == "" ).count()

```

```

+-----+-----+-----+-----+-----+
|Hours_Studied|Previous_Scores|Extracurricular_Activities|Sleep_Hours|Sample_Que
stion_Papers_Practiced|Performance_Index|
+-----+-----+-----+-----+-----+
|          0|          0|          0|          0|
0|          0|

```

```
+-----+-----+-----+-----+-----+
-----+-----+
```

chaînes vides :

[228]: 0

```
[229]: # .describe() donne les stats de base.
# .summary() est encore plus complet (ajoute les quartiles 25%, 50%, 75%)
df.select("Hours_Studied", "Previous_Scores", "Sleep_Hours",
↪ "Performance_Index").summary().show()
```

```
+-----+-----+-----+-----+-----+
--+
|summary|      Hours_Studied| Previous_Scores|      Sleep_Hours|
Performance_Index|
+-----+-----+-----+-----+-----+
--+
| count|           10000|           10000|           10000|
10000|
|  mean|           4.9929|           69.4457|           6.5306|
55.2248|
| stddev|2.5893087961856183|17.34315225104711|1.695862976517708|19.2125577994633
37|
|   min|              1|              40|              4|
10.0|
|   25%|              3|              54|              5|
40.0|
|   50%|              5|              69|              7|
55.0|
|   75%|              7|              85|              8|
71.0|
|   max|              9|              99|              9|
100.0|
+-----+-----+-----+-----+-----+
--+
```

3 DÉTECTION DES VALEURS ABERRANTES (OUTLIERS)

```
[230]: # On calcule les quartiles Q1 et Q3
quantiles = df.approxQuantile("Performance_Index", [0.25, 0.75], 0.05)
Q1 = quantiles[0]
Q3 = quantiles[1]
IQR = Q3 - Q1

# Définition des bornes
```

```

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df.filter((col("Performance_Index") < lower_bound) |
    ↪(col("Performance_Index") > upper_bound))

print(f"valeurs aberrantes de Performance_Index : {outliers.count()}")
if outliers.count() > 0:
    outliers.show()

```

valeurs aberrantes de Performance_Index : 0

4 Matrice de corrélation

```

[231]: import pyspark.ml

# Transformation texte -> binaire (Yes=1, No=0)
indexer = StringIndexer(inputCol="Extracurricular_Activities",
    ↪outputCol="Extracurricular_Bin")

df_corr = indexer.fit(df).transform(df)

[232]: # Liste de toutes les colonnes (Features + Cible)
all_cols = ["Hours_Studied", "Previous_Scores", "Extracurricular_Bin",
    ↪"Sleep_Hours", "Sample_Question_Papers_Practiced",
    ↪"Performance_Index"]

# Assemblage pour calcul matriciel
assembler_corr = VectorAssembler(inputCols=all_cols, outputCol="corr_features")
vector_df = assembler_corr.transform(df_corr).select("corr_features")

[233]: # Calcul de la matrice
matrix = Correlation.corr(vector_df, "corr_features").collect()[0][0]
print("Matrice de corrélation (Pearson) :")
print(matrix)

```

Matrice de corrélation (Pearson) :

```

DenseMatrix([[ 1.          , -0.01238992,  0.00387254,  0.0012452 ,  0.01746317,
                0.37373035],
              [-0.01238992,  1.          ,  0.00836932,  0.00594422,  0.00788803,
                0.91518914],
              [ 0.00387254,  0.00836932,  1.          , -0.02328364,  0.01310278,
                0.02452495],
              [ 0.0012452 ,  0.00594422, -0.02328364,  1.          ,  0.00399022,
                0.04810584],
              [ 0.01746317,  0.00788803,  0.01310278,  0.00399022,  1.          ,
                0.04326833],

```

```
[ 0.37373035, 0.91518914, 0.02452495, 0.04810584, 0.04326833,
1.      ]])
```

Hours_Studied vs Performance_Index (0.37) : Corrélation positive modérée. Plus on étudie, plus la note monte. C'est un moteur clé du modèle.

Previous_Scores vs Performance_Index (0.91) : Corrélation très forte. C'est ta variable la plus prédictive. Un étudiant qui a bien réussi par le passé a d'excellentes chances de réussir à nouveau.

Extracurricular_Bin vs Performance_Index (0.02) : Corrélation quasi nulle. Cela signifie que faire du sport ou de la musique n'impacte pas directement la note de manière linéaire, mais cela peut aider à stabiliser le profil.

5 PIPELINE DE TRAITEMENT (FEATURE ENGINEERING)

[234] :

```
# Assemblage des variables pour le modèle

feature_cols = ["Hours_Studied", "Previous_Scores", "Extracurricular_Bin",
                "Sleep_Hours", "Sample_Question_Papers_Practiced"]

assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
```

[] :

[235] :

```
# On teste le pipeline sur un échantillon pour vérifier la structure
test_pipeline = Pipeline(stages=[indexer, assembler])
df_prepared = test_pipeline.fit(df).transform(df)

print("Aperçu du vecteur 'features' (format attendu par le Random Forest) :")
df_prepared.select("Extracurricular_Activities", "Extracurricular_Bin",
                  ↪ "features").show(5, truncate=False)
```

Aperçu du vecteur 'features' (format attendu par le Random Forest) :

```
+-----+-----+-----+
|Extracurricular_Activities|Extracurricular_Bin|features          |
+-----+-----+-----+
|Yes                      |1.0                |[7.0,99.0,1.0,9.0,1.0]|
|No                       |0.0                |[4.0,82.0,0.0,4.0,2.0]|
|Yes                      |1.0                |[8.0,51.0,1.0,7.0,2.0]|
|Yes                      |1.0                |[5.0,52.0,1.0,5.0,2.0]|
|No                       |0.0                |[7.0,75.0,0.0,8.0,5.0]|
+-----+-----+-----+
```

only showing top 5 rows

6 ENTRAÎNEMENT DU MODÈLE RANDOM FOREST

```
[236]: # Split 80/20
train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)
```

```
[237]: # Modèle
rf = RandomForestRegressor(featuresCol="features",
    ↪labelCol="Performance_Index", numTrees=100)
```

```
[238]: # Pipeline final
full_pipeline = Pipeline(stages=[indexer, assembler, rf])
pipeline_model = full_pipeline.fit(train_df)
```

7 RÉSUMÉ DES MÉTRIQUES DE PERFORMANCE

```
[239]: from pyspark.ml.evaluation import RegressionEvaluator

# Génération des prédictions sur le jeu de test
predictions = pipeline_model.transform(test_df)
```

```
[240]: # On prépare l'évaluateur pour différentes métriques
evaluator = RegressionEvaluator(labelCol="Performance_Index",
    ↪predictionCol="prediction")

# Calcul des métriques clés
r2 = evaluator.evaluate(predictions, {evaluator.metricName: "r2"})
rmse = evaluator.evaluate(predictions, {evaluator.metricName: "rmse"})
mae = evaluator.evaluate(predictions, {evaluator.metricName: "mae"})
```

```
[241]: print(f"Coefficient de détermination ( $R^2$ ) : {r2:.4f}")
print(f"Erreur quadratique moyenne (RMSE) : {rmse:.4f}")
print(f"Erreur absolue moyenne (MAE) : {mae:.4f}")
```

```
Coefficient de détermination ( $R^2$ ) : 0.9151
Erreur quadratique moyenne (RMSE) : 5.5156
Erreur absolue moyenne (MAE) : 4.4152
```

8 IMPORTANCE DES VARIABLES

```
[242]: # Extraction du modèle Random Forest depuis le Pipeline
rf_model = pipeline_model.stages[-1]
importances = rf_model.featureImportances.toArray().tolist()
```

```
[243]: # Création d'un DataFrame Spark à partir des importances
# On combine nos noms de colonnes (feature_cols) avec leurs scores
feat_data = list(zip(feature_cols, importances))
```

```
feat_imp_spark_df = spark.createDataFrame(feet_data, ["Variable", "Importance"])
```

```
[244]: # Tri par importance décroissante (Calcul distribué)
feat_imp_sorted = feat_imp_spark_df.orderBy(col("Importance").desc())

print("TOP FEATURES:")
feat_imp_sorted.show()
```

TOP FEATURES:

Variable	Importance
Previous_Scores	0.8425961324123069
Hours_Studied	0.15033809465985712
Sleep_Hours	0.003476512629158...
Sample_Question_P...	0.002949033644814893
Extracurricular_Bin	6.40226653862594E-4

9 EXPORT VERS BIGQUERY

```
[245]: predictions = pipeline_model.transform(test_df)

# Paramètres GCP
project_id = "big-data-course-480109"
dataset_id = "performance_data"
table_id = "predictions_results"
temp_bucket = "perf-temp"

# Exportation
predictions.write.format("bigquery") \
    .option("temporaryGcsBucket", temp_bucket) \
    .option("table", f"{project_id}.{dataset_id}.{table_id}") \
    .mode("overwrite") \
    .save()
```

10 SAUVEGARDE DU MODÈLE

```
[246]: # Chemin sur ton bucket GCS
model_path = "gs://student-performance-exam/models/student_rf_model"

# .write().overwrite() permet de mettre à jour le modèle si tu relances le code
pipeline_model.write().overwrite().save(model_path)
```

11 CHARGEMENT ET TEST DE PRÉDICTION

```
[247]: loaded_model = PipelineModel.load(model_path)

# Création d'une donnée de test (Nouvel étudiant)

new_student_data = [
    (6, 85, "Yes", 7, 4) # Heures_Etude, Scores_Precedents, Extra, Sommeil,
    ↪ Sujets
]

new_student_df = spark.createDataFrame(new_student_data, [
    "Hours_Studied", "Previous_Scores", "Extracurricular_Activities",
    "Sleep_Hours", "Sample_Question_Papers_Practiced"
])
```

```
[248]: # Application du modèle
# Le pipeline va automatiquement transformer "Yes" en 1 et assembler le vecteur
    ↪ features
prediction_result = loaded_model.transform(new_student_df)

print(" NOUVEL ÉTUDIANT RESUL:")
prediction_result.select(
    "Hours_Studied",
    "Previous_Scores",
    "Extracurricular_Activities",
    "Sleep_Hours", "Sample_Question_Papers_Practiced",
    "prediction"
).show()
```

```
NOUVEL ÉTUDIANT RESUL:
+-----+-----+-----+-----+-----+
|Hours_Studied|Previous_Scores|Extracurricular_Activities|Sleep_Hours|Sample_Que
stion_Papers_Practiced|prediction|
+-----+-----+-----+-----+-----+
|          6|          85|                Yes|          7|
4|74.44370878125524|
```

12 conclusion Modele

L'analyse de ce projet démontre la robustesse du modèle de régression Random Forest pour prédire l'index de performance des étudiants avec une précision remarquable. Les métriques obtenues, notamment le coefficient R^2 proche de l'unité, confirment que les variables sélectionnées expliquent bien la variance des notes. La faiblesse des erreurs moyennes (RMSE et MAE) souligne la fiabilité des prédictions, validant ainsi la pertinence du choix algorithmique pour ce type de problématique éducative.

L'étude de l'importance des variables révèle une hiérarchie structurelle au sein des facteurs de réussite. Il apparaît clairement que le passé académique, représenté par les scores précédents, constitue le moteur principal de la performance future. L'investissement temporel dans l'étude suit de près, affirmant la corrélation positive entre l'effort fourni et les résultats obtenus. En revanche, les variables telles que le sommeil, la pratique de sujets d'examen ou les activités extra-scolaires, bien qu'utiles à l'équilibre général, n'exercent qu'une influence marginale sur la prédiction finale de la note.

13 archi

Sur le plan technique, l'architecture déployée illustre parfaitement les avantages d'un écosystème Big Data moderne. L'utilisation d'Apache Spark a permis un traitement distribué efficace, affranchi des limites de mémoire traditionnelles, tandis que l'intégration avec BigQuery et Looker Studio a facilité une transition fluide entre le calcul brut et l'analyse décisionnelle. Enfin, la sauvegarde du pipeline complet garantit la pérennité de la solution, permettant un déploiement opérationnel capable de fournir des prédictions immédiates pour accompagner les étudiants vers la réussite.

[]: